AGENTIC AI

Merchandising AI

A Multi-Agent System for Non-Purchaser Feedback Analytics

Architecture · Topology · Agents · Tools · Evaluation · Observability

PRESENTED BY
Sundar Ram Subramanian

DATE
May 16, 2026

OVERVIEW

Project at a Glance

THE BUSINESS PROBLEM

Six-store jewelry chain (X1–X6). Salesmen capture free-text reasons when a customer leaves without buying. ~1,200 records.

Merchandising team needs to know:
"What should we stock more at X2?" · "Compare size issues across X1 and X4." · "What's the top reason at X3 yesterday?"

THE ARTIFACT

Production-grade multi-agent chat assistant. Natural-language questions in. Grounded, source-cited, SQL-backed answers out.

9 specialised LLM agents · 11 deterministic tools · 1 rule-based Python orchestrator · 1 audit log.

9	11	2	0.0	≈$0.02
LLM agents (5 reasoning + 4 reflection)	deterministic tools	model tiers (Sonnet 4.6 + Haiku 4.5)	Temperature. Highly deterministic agents	happy-case cost per SQL turn

Two surfaces.
Chat - free-form Q&A driven by the agent pipeline.
Recommendations - one-click deterministic report computed in pandas, identical every time.

DESIGN RATIONALE

Why a Multi-Agent System is the Right Tool?

"What should we stock more at X1?" looks like one question. It is actually six obligations a single LLM call cannot satisfy simultaneously:

1 Translate

Map colloquial business phrasing into a precise multi-dimensional SQL aggregation.

2 Execute safely

Run that SQL against production data with SELECT-only / banned-keyword guards.

3 Verify semantics

Check that the rows returned actually answer the question that was asked.

4 Compose prose

Write a markdown answer that cites only numbers verifiable against the data.

5 Visualise (if it helps)

Render an optional chart whose code and image both pass review.

6 Surface reasoning

Expose every step so a sceptical stakeholder can audit the answer end-to-end.

Takeaway. Task decomposition and splitting of works across specialised agents whose outputs are **independently reviewed** is the smallest design that satisfies all six obligations. A single prompt or a plain RAG retrieval cannot.

CROSS-CUTTING PRINCIPLES

Five Foundational Design Choices

These five choices touch every layer of the system. They are the doctrines the rest of the architecture flows from.

1

Determinism over creativity

All 9 agents run at temperature 0.0. Writer was dropped from 0.2 → 0.0 after the same SQL value of 41 yielded “approximately 40,” “exactly 41,” and fabricated subset sums on different retries.

2

Rule-based Orchestrator, not an autonomous loop

Control plane is pure Python. Agents emit structured text in XML tags or JSON; the Orchestrator parses and routes. No LLM ever decides what runs next.

3

Closed enums everywhere

Reviewer verdicts, Planner path, Supervisor action, Topic Classifier output - all enum-checked in both prompt and Python. Out-of-enum output counts against the retry budget.

4

Hard-block over soft-block on numeric grounding

If the Writer cannot ground every number in two attempts, the system ships an explicit refusal, not unverified prose with the wrong subtotal.

5

Schema vocabulary discipline

Renamed attribute_type → non_purchase_type because the word “attributes” collided with another column and caused the LLM to disambiguate incorrectly. Schema-level naming clarity beats prompt-level scaffolding.

HIGH-LEVEL ARCHITECTURE

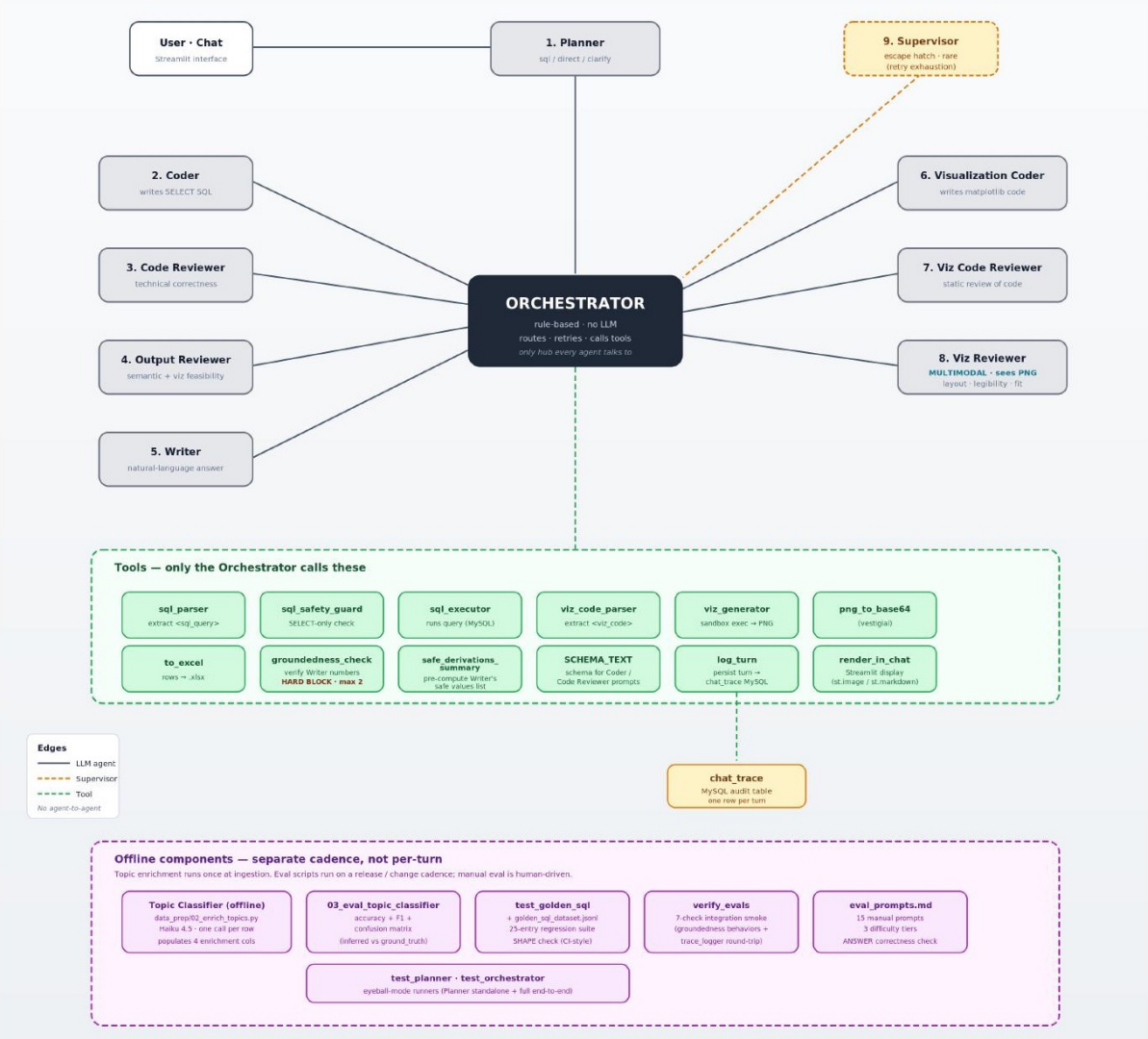
The Eight-Layer Stack

Each layer has a single concern. Each layer talks only to its neighbours. The Orchestrator is the only component permitted to call agents or tools.

8	**Observability & Interface**	Streamlit UI · StepEvent · chat_trace surfacing
7	**Tooling & Validation**	sql_parser · sql_safety_guard · sql_executor · viz tools · groundedness · log_turn
6	**Orchestration & Recovery**	Rule-based control plane · retry budgets · Supervisor escape hatch
5	**Reflection**	Code Reviewer · Output Reviewer · Viz Code Reviewer · Viz Reviewer (multimodal)
4	**Reasoning**	Planner · Coder · Writer · Viz Coder · Supervisor
3	**Memory**	Streamlit session state · 8-message look-back · resolved_question mechanism
2	**Knowledge**	Offline topic-enrichment pipeline · SCHEMA_TEXT constant
1	**Data**	MySQL · non_purchasers_feedback (~1,200 rows) · chat_trace audit table

COMPONENT CALL GRAPH

A Single Hub Topology



WHY THIS SHAPE

Every agent is connected only to the Orchestrator.

Every tool is connected only to the Orchestrator.

No agent ever calls another agent. No agent ever calls a tool.

The hub-and-spoke shape is the design statement.

FIVE CONCRETE BENEFITS

single place to set retry budgets

log every step

apply safety guards

audit every tool call

read control flow top-to-bottom in one Python file

THE MOST CONSEQUENTIAL CHOICE

Single-Hub Orchestrator vs MCP / Tool-Use Loop

AUTONOMOUS AGENT LOOP-REJECTED

MCP servers · LangGraph · AutoGen · Anthropic tool_use

Single LLM decides at each step which tool to invoke next.

- Flexible - agent navigates problems it was not programmed for
- Worst-case cost is unbounded
- Retry budgets are hard to enforce
- Tracing requires decoding the LLM's tool-use rationale
- Cost ceilings drift as the model decides its own loop length

RULE-BASED PYTHON ORCHESTRATOR-CHOSEN

agents/orchestrator.py · ~679 lines · zero LLM in routing logic

LLMs emit structured text. Python parses and dispatches.

- Worst-case cost is bounded by retry budgets
- Retry budgets enforced by Python, not by the LLM
- Every tool call goes through one auditable control-flow point
- No tool-use round trips inflate cost
- Debugging is reading Python, not decoding LLM rationale

Trade-off accepted: less flexibility on never-seen problems. **Trade-off rejected:** unbounded cost, opaque control flow, and unreliable retry enforcement. For a focused merchandising chat where the same control flow applies to every turn, autonomy is the wrong trade.

Two Deliberate Non-Choices

NO VECTOR DATABASE · NO RAG

No embedding model. No similarity index. No retrieval-augmented generation.

Why?

- Data is small: ~1,200 rows of structured feedback
- Schema is fixed: known columns, known types
- Questions are quantitative aggregations, not semantic search

Right primitive: **SQL with a strongly-typed schema.**

When to reconsider: scope widening to free-text Q&A over policy documents or product PDFs.

NO LANGCHAIN · NO LANGGRAPH

Neither LangChain nor LangGraph is a dependency. orchestrator.py is ~679 lines of plain Python.

Trade-off accepted:

Adding a new agent requires editing the orchestrator file rather than declaring a node in a graph.

Benefits gained:

- Total visibility into control flow: every retry, every escape hatch, every step event visible inline
- Zero exposure to framework breakage when an upstream library tweaks its public interface
- Lower cognitive load for the next engineer to read the codebase

Both non-choices are reversible: the system would absorb a vector store or a framework if the problem shape changed. This problem shape does not require them.

ANTHROPIC CLAUDE · SONNET 4.6 + HAIKU 4.5

Model Strategy — A Deliberate Two-Tier Allocation

Sonnet 4.6 is used only where it is structurally required. Haiku 4.5 is roughly 3–5× cheaper and used everywhere else, where it delivers Sonnet-acceptable quality.

Agent	Model	Temp	Why this model
Coder	Sonnet 4.6	0.0	Hardest task. Haiku failed on MySQL window-functions and on the non_purchase_type vs attribute_value disambiguation.
Code Reviewer	Sonnet 4.6	0.0	Must match the Coder's tier or it rubber-stamps subtle errors. Tier-matching is what makes reasoning-vs-SQL verification work.
Planner	Haiku 4.5	0.0	Routes the turn into sql / direct / clarify. A simple classification task.
Output Reviewer	Haiku 4.5	0.0	Binary verdict on whether rows answer the question. Haiku is sufficient.
Writer	Haiku 4.5	0.0	Composes prose from a pre-computed safe-values list. Arithmetic is taken off the LLM.
Viz Coder	Haiku 4.5	0.0	Writes short, pattern-based matplotlib snippets. Well-bounded task.
Viz Code Reviewer	Haiku 4.5	0.0	Static safety check for banned identifiers and missing labels.
Viz Reviewer	Haiku 4.5 vision	0.0	Must SEE the rendered PNG. Requires multimodal input natively.
Supervisor	Haiku 4.5	0.0	Picks one of four recovery actions from the step trace. A small decision.

*All agents run at temp 0 to maximise determinism. **Sonnet** = SQL-quality bottleneck only. **Haiku vision** = the one agent that must inspect an image. Everything else = standard Haiku.*

THE TWO SPECIFIC FAILURE MODES SONNET FIXED

Why Sonnet 4.6 for the Coder + Code Reviewer

Earlier iterations ran the Coder on Haiku 4.5 to save cost. Two specific failure modes drove the upgrade.

FAILURE MODE 1 — WINDOW FUNCTIONS

MySQL window-function syntax required for top-N within each group.

```
ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)
```

Haiku 4.5: consistently mis-syntaxed the PARTITION BY clause, omitted the ORDER BY, or fell back to nested sub-queries that returned the wrong row count.

Sonnet 4.6: handles it reliably under the same prompt.

FAILURE MODE 2 — DOMAIN VOCABULARY

Disambiguating non_purchase_type vs attribute_value — central to the merchandising domain.

```
non_purchase_type ∈  {design, size, color, weight, price,
customization, ...}
attribute_value ∈  {'rose gold', 'size 8', 'star design', ...}
```

Haiku 4.5: filtered on the wrong column on compound questions like *"top design attributes at X1"*

Sonnet 4.6: picks the right column under the same prompt.

THE REVIEWER MUST MATCH THE CODER

A Haiku-tier reviewer cannot catch a Sonnet-tier Coder's subtle mistakes. Tier-matching is what makes the reasoning-vs-SQL verification work in practice

KNOWLEDGE LAYER

Data Layer & Closed-Enum Discipline

One MySQL table. Nine captured columns + four LLM-enriched columns. Enrichment runs once offline and not per chat turn.

THE SCHEMA

Captured at the storefront (9 cols)

`feedback_id` · `visit_date` · `store_code` · `customer_name` · `customer_email` · `user_category` · `product_looking_for` · `reason_for_non_purchase` · `ground_truth_topic`

Enriched offline by LLM Topic Classifier (4 cols)

`inferred_topic` · `topic_confidence` · `non_purchase_type` · `attribute_value`

Why offline? Pattern-matching free-text reasons inside every chat turn would be fragile and slow. Persisting clean structured columns lets the Coder write aggregations against enums.

CLOSED ENUMS — DUAL ENFORCEMENT

Validated in the prompt AND in Python. Out-of-enum output counts against the retry budget.

inferred_topic	→ non_purchase_type
Design Unavailable	`design`
Size Unavailable	`size`
Stock Unavailable	`stock`
Price Too High	`price`
Quality Concerns	`none`
Weight Concerns	`weight`
Color/Finish Mismatch	`color`
Customization Not Offered	`customization`
Sales Service	`service`
Others	`none`

Vocabulary discipline goes back to the schema. Column was originally `attribute_type` — the word “attributes” collided with another column. Renaming it to `non_purchase_type` prevented an entire class of disambiguation errors.

TWO HORIZONS, ONE RESOLUTION POINT

Memory & Context Management

SHORT-TERM · PER-SESSION

8-message look-back at the Planner only

The Planner sees `history[-8:]` and emits a `resolved_question` which is a self-contained string with context baked in.

Every downstream agent (Coder, Output Reviewer, Writer, Viz Coder) reads the resolved question but never the raw user message.

Why 8?

- Empirically, follow-ups point back 1–2 turns; depth > 4 rarely catches new references
- Keeps Planner prompt small (~1–2K tokens total)
- Caps total token cost to O(n) instead of O(n^2) on long sessions

LONG-TERM · CROSS-SESSION

chat_trace MySQL audit table

Every chat turn → one row. Derived columns (path, retry_count, supervisor_invoked, groundedness_warned) + full StepEvent trace as JSON.

Why denormalised?

- Common queries are “give me all turns where X happened”. The derived columns answer them without JSON-path parsing
- Full JSON trace preserves complete detail for the rarer “give me every step in turn Y” case

Logging is wrapped in two layers of try/except:
MySQL outage cannot affect the chat response that has already been returned.

5 REASONING + 4 REFLECTION

The Nine-Agent Roster

Each reasoning agent has a paired reflection agent. Cross-review, not self-critique. All temp 0, all closed-enum verdicts.

REASONING AGENTS (5)

1 Planner — Haiku
Routes turn into sql / direct / clarify · resolves history

2 Coder — Sonnet
Writes <reasoning> + <sql_query> — the SQL bottleneck

5 Writer — Haiku
Composes markdown answer from safe-values list

6 Viz Coder — Haiku
Writes short matplotlib snippet for the sandbox

9 Supervisor — Haiku
Picks recovery action when a retry budget exhausts

REFLECTION AGENTS (4)

3 Code Reviewer — Sonnet
Verifies SQL technical correctness + reasoning-vs-SQL consistency

4 Output Reviewer — Haiku
Decides if rows answer the question · flags viz_applies

7 Viz Code Reviewer — Haiku
Static safety check - banned identifiers, missing labels

8 Viz Reviewer — Haiku vis
MULTIMODAL - judges the rendered PNG: layout · legibility · fit

+ 1 offline agent: Topic Classifier (Haiku 4.5, one-time, ~1,200 rows, 10-topic closed enum) - runs at ingestion, not per turn

Eleven Deterministic Tools

Every tool is pure Python. Every tool receives pre-parsed, pre-validated data but never raw model text. Every tool is called exclusively by the Orchestrator.

SQL
sql_parser
Extract <sql_query> + <reasoning> blocks from Coder output

SQL
sql_safety_guard
SELECT-only + banned-keyword + multi-statement check

SQL
sql_executor
Run SQL against MySQL · cap 500 rows

VIZ
viz_code_parser
Extract Python from <viz_code> tags

VIZ
viz_generator
Sandboxed exec of matplotlib code · whitelist __builtins

VIZ
png_to_base64
Encode rendered PNG bytes for multimodal call

I/O
to_excel
openpyxl-backed .xlsx bytes for download affordance

VERIFY
groundedness_check
Regex-based numeric verification · HARD BLOCK

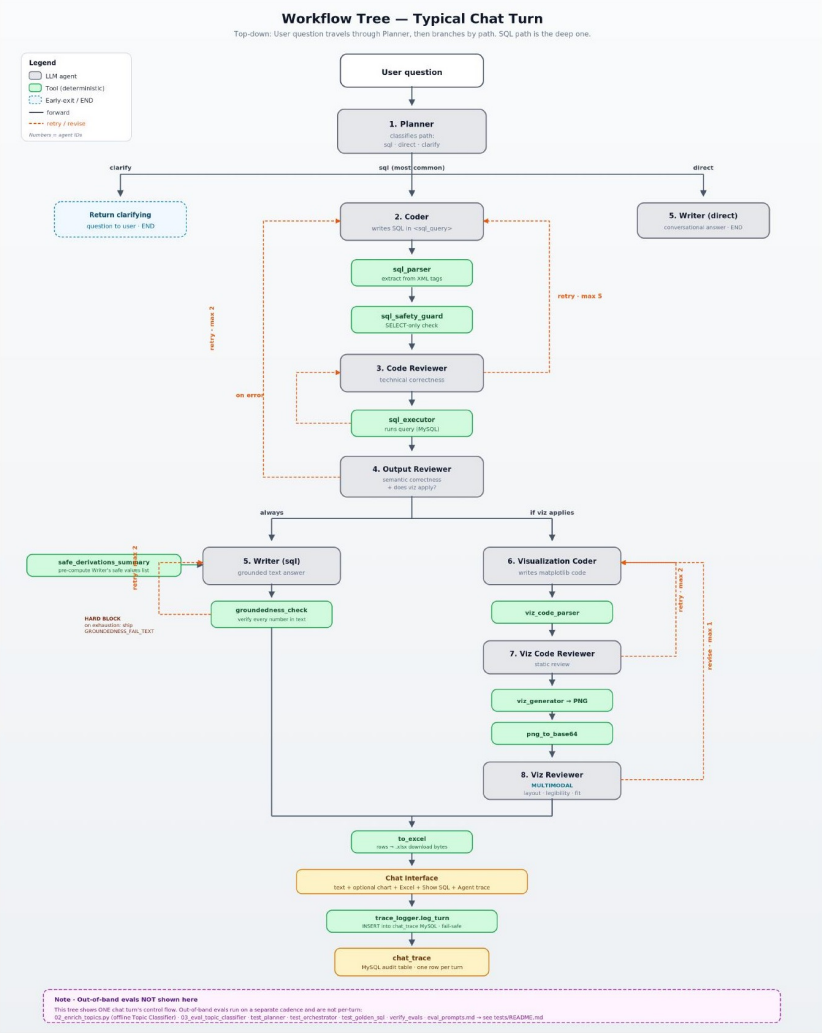
VERIFY
safe_derivations_summary
Pre-compute the Writer's safe-values list

OBS
log_turn / ensure_table
Persist one row to chat_trace · idempotent schema

META
SCHEMA_TEXT / get_schema_for_prompt
Canonical schema as a string constant

Why deterministic, not agent-driven tool-use? Tools receive Python values, not raw model text. Each tool is trivially unit-testable and free of any model-specific assumption. This is what makes worst-case cost bounded.

End-to-End Workflow — One SQL-Path Chat Turn



THE FIVE NESTED LOOPS

5 — Inner SQL loop
Coder ↔ Code Reviewer ↔ sql_executor

2 — Output review loop
Output Reviewer ↔ Coder

2 — Writer grounding loop
Writer ↔ groundedness_check

2 — Viz code loop
Viz Coder ↔ Viz Code Reviewer

1 — Viz revise loop
Viz Reviewer (multimodal)

Three loops escalate to the Supervisor on exhaustion (SQL strategy can plausibly change). Two do not (Writer grounding → hard-block refusal; Viz pipeline → drop chart, ship text).

WORST-CASE COST IS BOUNDED BY DESIGN

Retry Budgets & Supervisor Recovery

Every retry budget lives as a module-level constant at the top of orchestrator.py. No magic numbers buried in functions.

Loop	Max	Exhaustion behaviour
Coder ↔ Code Reviewer ↔ sql_executor	5	Escalate to Supervisor
Output Reviewer ↔ Coder	2	Escalate to Supervisor
Writer ↔ groundedness_check	2	**Ship GROUNDEDNESS_FAIL_TEXT (hard-block)**
Viz Coder ↔ Viz Code Reviewer	2	Drop chart · ship text-only
Viz Reviewer revise round	1	Ship last PNG or drop

WHEN A SQL LOOP EXHAUSTS: SUPERVISOR'S FOUR ACTIONS

`abort_gracefully`
Question is genuinely unanswerable. Ship a polite explanation.

`retry_with_strategy`
Propose a fundamentally different SQL approach. One-shot only.

`ship_partial`
Useful result exists. Ship it with a caveat about limits.

`ask_user`
Question is ambiguous. Return a focused clarifying question.

Hard-block vs soft-block is deliberate. ***Writer grounding → hard-block:*** *a strategy change does not help; refuse with a fixed message.* ***SQL loops → soft-block to Supervisor:*** *different SQL might actually answer.*

Three Coordinated CoT Scaffolds

Chain-of-thought is not a single prompt trick in this system. It is a triplet of mechanisms that guard a distinct failure surface each. Removing any one of them produces measurable regression on the manual eval suite.

1 Coder's <reasoning> block

GUARDS AGAINST

Coder writes SQL that ignores its own intent

MECHANISM

Before any SQL is emitted, the Coder is required to produce four named lines: Dimensions / Filters / Aggregation / Completeness. Once the structure is committed in text, the model's own consistency bias makes it linguistically jarring to contradict it in the SQL that follows.

LIVES IN

Coder agent

2 Pre-computed safe-values list

GUARDS AGAINST

Writer fabricates a subtotal that misses the candidate set

MECHANISM

After SQL runs, safe_derivations_summary computes in plain Python and every legitimate numeric derivation: grand totals, top-N partial sums, per-group subtotals, per-group row counts. The Writer is instructed to pick numbers from this list rather than compute its own.

LIVES IN

Orchestrator (Python) → Writer agent

3 Reasoning-vs-SQL verification

GUARDS AGAINST

Plausible-looking SQL that quietly contradicts the reasoning

MECHANISM

The Code Reviewer reads both the <reasoning> block and the SQL, and verifies one against the other. Reasoning says “three dimensions” but SQL groups by two → retry. Reasoning says “no LIMIT” but SQL has LIMIT 30 → retry.

LIVES IN

Code Reviewer agent

Scaffold 1 prevents missed dimensions. Scaffold 2 prevents fabricated subtotals. Scaffold 3 catches Scaffold 1 being ignored. **Defence in depth, by design.**

WHY REVIEWER ≠ GENERATOR

Reflection: Cross-Review, Not Self-Critique

Each reviewer's verdict is a closed enum and each reviewer's feedback is a concise, concrete suggestion. The reasoning-vs-reflection split was chosen for two specific reasons.

REASON 1 — DILUTED ATTENTION

Prompting one agent to both produce and critique its own output dilutes the system prompt and produces measurably worse criticism. The generator's prior toward its own output overwhelms the critic's job.

REASON 2 — CLEAN BUDGET BOUNDARY

The reviewer's verdict is what determines whether the loop continues or terminates. That decision belongs in its own agent with its own narrow prompt and its own line in the retry-budget table.

FOUR DEDICATED REVIEWERS — EACH WITH ITS OWN GUARD

Code Reviewer

Guards against
SQL correctness + reasoning consistency

Verdict enum
`{ ok · retry }`

Output Reviewer

Guards against
Do the rows answer the question?

Verdict enum
`{ ok · retry } + viz_applies`

Viz Code Reviewer

Guards against
Static safety check on viz code

Verdict enum
`{ ok · retry }`

Viz Reviewer (mm)

Guards against
Multimodal: judges the rendered PNG

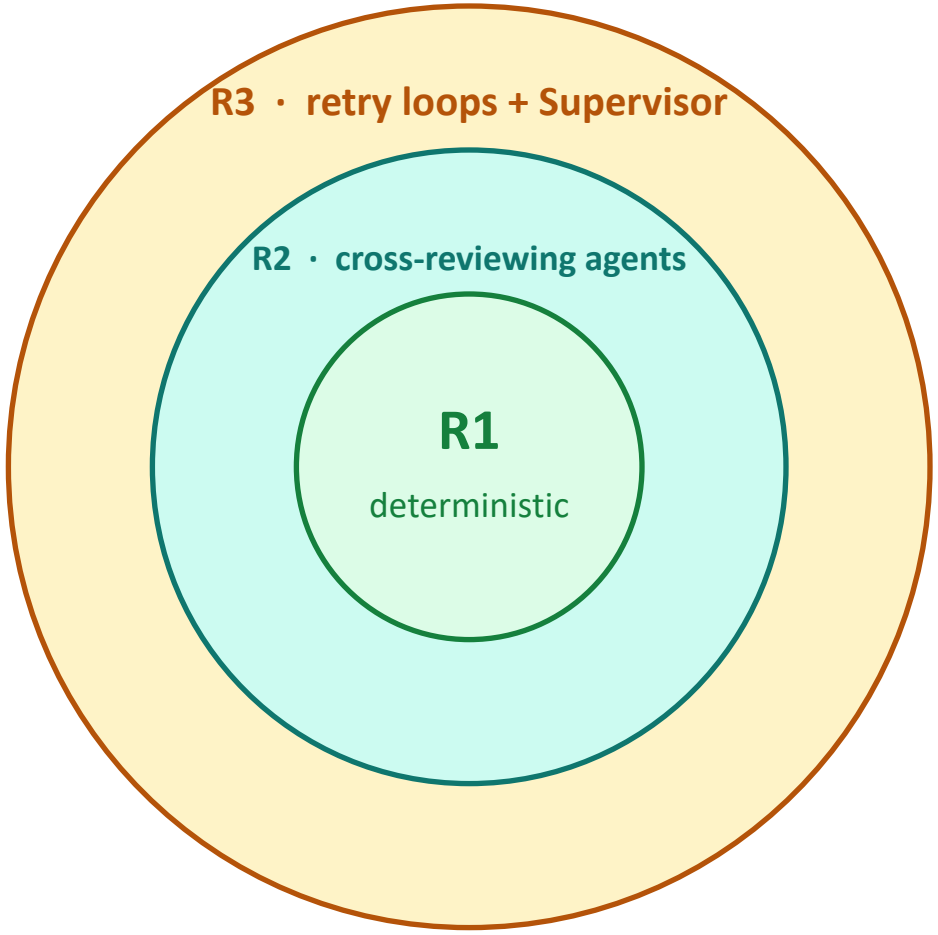
Verdict enum
`{ ok · revise · drop }`

Why not a single critic at the end? By the time a final critic fires, the system has already paid for Coder, sql_executor, Writer, and the viz pipeline. **Critique at each stage means rejection happens at the cheapest point in the pipeline at which the error is detectable.**

IN-BAND · EVERY TURN

Evaluation: Three Concentric Runtime Rings

Each ring catches a different class of failure. Outer rings handle what inner rings cannot see.



Ring 1

Deterministic guards

sql_parser · sql_safety_guard · viz_code_parser · viz_generator sandbox · groundedness_check · closed-enum validators on every structured output.

Cannot lie — pure Python, unit-testable.

Ring 2

Cross-reviewing LLM agents

Code Reviewer · Output Reviewer · Viz Code Reviewer · Viz Reviewer. These are LLMs and can occasionally be wrong.

Catch subtle errors the deterministic guards cannot see.

Ring 3

Retry loops + Supervisor escape hatch

Every reviewer veto bounces back to the reasoning agent with feedback as a hint. Bounded retries protect against infinite loops.

Supervisor recovers when SQL loops exhaust.

RUN ON A CADENCE · NOT PER TURN

Four Out-of-Band Evaluation Suites

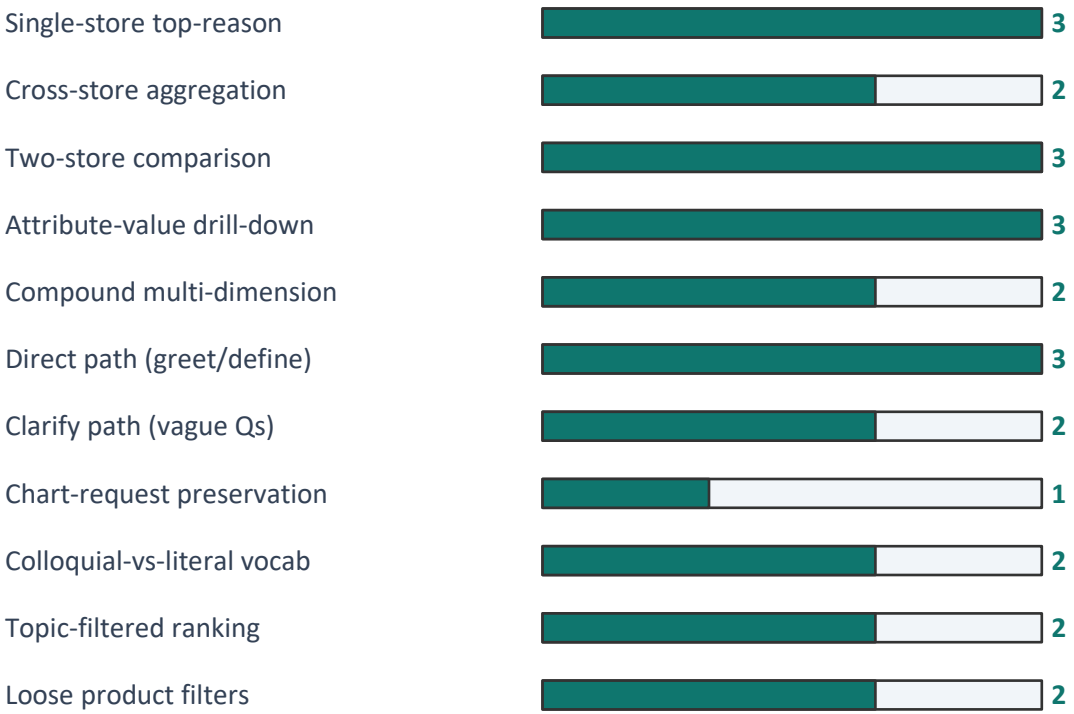
Each suite plays a distinct role in the trust-but-verify story. Running all four after any prompt or model change is the only reliable way to confirm no regression.

01	**Integration smoke test** `verify_evals.py` 7 checks confirming groundedness behaviours + trace_logger round-trip	**CATCHES** Wiring breaks · regressions in the safety-rail seam between agents and tools	**CADENCE** After any code or prompt change
02	**Q-to-SQL regression suite** `test_golden_sql.py + dataset.jsonl` 26 entries · automated SHAPE check (path · SQL tokens · row-count bounds)	**CATCHES** Planner path drift · forbidden columns · missing GROUP BY · removed LIMIT	**CADENCE** CI-style on every release
03	**Manual answer-correctness suite** `eval_prompts.md` 15 prompts at 3 difficulty tiers · numbers compared against Excel pivots	**CATCHES** Subtle aggregation errors that pass the shape check but produce wrong numbers	**CADENCE** Human-driven · before any major release
04	**Topic classifier accuracy** `03_eval_topic_classifier.py` Precision · Recall · F1 per topic + full confusion matrix + low-confidence triage	**CATCHES** Drift in the offline enrichment that quietly biases all downstream aggregation	**CADENCE** After any enrichment-prompt change

Why both a shape check and an answer-correctness check? *A row-count drift can pass the shape check; only the manual numeric comparison catches it. Conversely, a refactor that breaks path-routing is caught instantly by the shape check. Both layers are needed.*

Eval Suite Deep Dive

GOLDEN SQL DATASET — 25 ENTRIES



Each entry asserts: (a) Planner picks the expected path · (b) SQL contains required tokens, avoids forbidden ones · (c) row count is within bounds. Shape check and not answer-correctness.

TOPIC CLASSIFIER METRICS

Compared against synthetic ground_truth_topic. Per-topic + overall metrics + confusion matrix.

Precision(t)	`= TP(t) / ( TP(t) + FP(t) )`
Recall(t)	`= TP(t) / ( TP(t) + FN(t) )`
F1(t)	`= 2 · Precision(t) · Recall(t) / ( Precision(t) + Recall(t) )`
Accuracy	`= Σ_t TP(t) / N_total`

Script also prints lowest-confidence mistakes — a regression often manifests as a small number of high-confidence wrong predictions, more informative than the overall accuracy number.

Topic Classifier is the most consequential bias surface in the system — a topic systematically under-classified at one store would silently misdirect merchandising decisions. Per-topic F1 is monitored individually, not blended into a single accuracy figure.

THE HIGHEST-LEVERAGE QUALITY INTERVENTION

The Groundedness Check : Why Regex Beats LLM-as-Judge

For a quantitative business-analytics use case, numeric hallucination is the most damaging failure the system can produce. A fabricated count becomes a wrong purchase order.

HOW IT WORKS

Every numeric token in the Writer's prose is extracted by regex and verified against a broad candidate set derived from the SQL result rows.

individual cells	**numbers in string labels**	**per-col grand totals**	**per-row column %**	**per-group subtotals**	**per-group row counts**	**top-N partial sums (N=2,3,4,5)**

REGEX CHECK — CHOSEN

- Deterministic: same input, same verdict every time
- Zero additional cost: no extra API call per turn
- Trivially explainable to a stakeholder
- Structurally incapable of being talked into letting a bug through

LLM-AS-JUDGE — REJECTED

- Stochastic: same input may produce different verdicts
- Adds another API call per turn
- Requires its own eval harness to confirm the judge is correct
- Harder to explain: “the model said so” isn't an audit trail

Trade-off accepted: the regex cannot verify semantic attribution, but it can confirm “41 appears in the rows” but not “41 was attributed to the right group”. **The Output Reviewer catches misattribution. The layering is intentional.**

EVERY STEP. EVERY TURN. PERSISTED TO MYSQL.

Observability & the chat_trace Audit Log

CHAT_TRACE SCHEMA · ONE ROW PER TURN

Column	Type	Purpose	Idx
trace_id	BIGINT PK	Auto-incrementing turn id	
created_at	DATETIME	Insertion timestamp	◆
path	VARCHAR(16)	sql / direct / clarify	◆
resolved_question	TEXT	Planner's context-resolved Q	
final_sql	TEXT	SQL that ran (NULL on non-sql)	
has_chart	BOOL	Was a chart rendered?	
has_excel	BOOL	Excel download attached?	
supervisor_invoked	BOOL	Was the Supervisor reached?	◆
groundedness_warned	BOOL	Hard-block fired?	◆
step_count	INT	Number of StepEvents this turn	
retry_count	INT	Steps with status=retry	
answer_text	MEDIUMTEXT	Final user-facing markdown	
steps_json	MEDIUMTEXT	Full step trace as JSON	

◆ *indexed for common queries*

Denormalised intentionally, common queries are "give me all turns where X happened", not "every step in turn Y".

STEPEVENT · THE ATOM OF TRACING

agent	Which agent or tool acted
status	start · ok · retry · fail
summary	One-line human-readable
detail	Agent-specific structured dict

IN-UI TRACE SURFACING

Every chat response carries four expandable panels:

- Answer markdown - inline
- Show SQL - executed query
- Show chart code - matplotlib snippet
- How I got this answer - full StepEvent trace

Sceptical user audits the system without ever leaving the chat surface.

HONEST SELF-ASSESSMENT, NOT A WIKI-PAGE POLICY

Ethics · PII - Incidental vs Designed Protection

The system has two PII columns: customer_name and customer_email. PII protection is incidental: a behavioural default, not a structural guarantee.

CURRENT — INCIDENTAL PROTECTION

Four behavioural defaults keep PII out of typical answers:

- Coder corpus is aggregation-shaped - no example mentions PII columns
- Planner resolves into aggregation phrasing, never row-listing
- Writer template composes grouped summaries, not row lists
- Anthropic RLHF provides a thin backstop against gratuitous PII

Brittle. *A directly phrased query - "list the names of customers who complained about size at X1" – might exposes the gap.*

FUTURE — DESIGNED PROTECTION

Two ~100-line changes, both unit-testable:

- PII-scrub wrapper around sql_executor - strips customer_name and customer_email from every result row before it reaches any agent
- Writer-prompt rule forbidding citation of individual customers - defence-in-depth even if scrub is changed

Robust. *Agents never receive PII columns - the SQL itself cannot exfiltrate them, regardless of phrasing.*

WHEN THE UPGRADE BECOMES MANDATORY

Any of: external user access · internal access beyond the merch team · regulatory audit · use-case expansion to individual-customer questions (loyalty programmes, single-customer journeys).

PER-TURN LATENCY + PER-TURN TOKEN SPEND

Performance & Cost : Bounded by Design

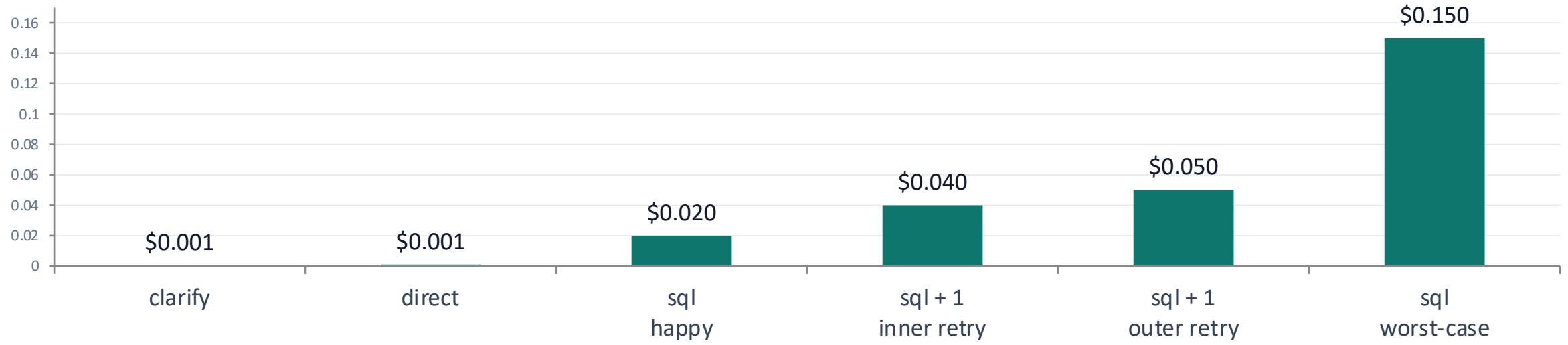
15–20s
SQL happy case
(1 Coder · 1 OR · 1 Writer)

30–60s
SQL with retries
(1–2 retry attempts)

2–5s
Direct path
(conversational only)

1–2s
Clarify path
(planner returns clarifying Q)

PER-TURN COST DECOMPOSITION · ANTHROPIC PRICING



Worst-case cost is bounded. Sub-dollar even in the worst case because retry budgets are bounded. This is a direct consequence of the rule-based Orchestrator choice. **An autonomous-loop design could not make this guarantee.**

WHAT'S NOT YET BUILT — AND WHY

Known Limitations & Future Directions

KNOWN LIMITATIONS

- Groundedness verifies values, not attribution - wrong-group attribution can pass
- Planner is a single point of failure for history resolution
- Output Reviewer at Haiku can occasionally rubber-stamp subtle mismatches
- PII is not guarded - incidental protection only (see Slide 24)
- Viz sandbox is dev-grade, in-process exec, not subprocess-isolated
- Tests are CI-ready but no CI workflow committed yet
- Golden SQL set is starter-size (25 entries), should grow with new failure modes
- No prompt-version hash stored in chat_trace, git-only traceability
- Sonnet at temp 0 is not strictly bit-deterministic, identical-replay is imperfect
- Cost rose ~3–5× with the Sonnet upgrade, worth re-checking quarterly

FUTURE DIRECTIONS

- CI integration: wire golden_sql + verify_evals into GitHub Actions on every PR
- Prompt-version tracking : store prompt-file hashes in chat_trace.steps_json
- PII redaction: sql_executor wrapper that scrubs customer_name and customer_email
- Subprocess-isolated viz sandbox: for production multi-tenancy
- Vector store (only) if scope widens to free-text Q&A over policy/PDF documents
- Per-turn cost ceiling: hard token-budget check that aborts a runaway turn
- Promote Output Reviewer to Sonnet 4.6 once cost permits.

THE ARCHITECTURAL THESIS

Match commitments to the problem.

A focused multi-agent system, built with deliberate rule-based orchestration and aggressive layered evaluation, can reliably translate natural-language business questions into grounded SQL answers at a bounded cost-per-turn.

Every architectural commitment in this system: ***single-hub orchestration, two-tier model selection, CoT scaffolding triplet, hard-block groundedness, end-to-end observability*** were made because the specific business problem rewards predictability over flexibility.

Every answer can be audited

Every retry can be bounded

Every regression can be detected

Thank you · Questions welcome